NuFast-LBL User Guide

Peter B. Denton[1, *] and Stephen J. Parke[2, †]

[1] *High Energy Theory Group, Physics Department Brookhaven National Laboratory, Upton, NY 11973, USA*

[2] *Theoretical Physics Department, Fermi National Accelerator Laboratory, Batavia, IL 60510, USA*

2405.02400

v2.0.0

CONTENTS

I. INTRODUCTION

Computing neutrino oscillation probabilities in matter is a computationally expensive problem often dominating the computational cost of increasingly expensive fits to increasingly precise neutrino oscillation data. The code presented here, NuFast-LBL, leverages the algorithm developed in [1] and previous papers. It aims to address this problem for cases when the density profile the neutrinos are traveling through is effectively constant. DUNE is the long-baseline (LBL) neutrino oscillation experiment that travels the deepest into the crust and thus experiences the largest variation in the matter effect and is also the LBL experiment that is most strongly impacted by the matter effect, the small variation due to traveling through different parts of the Earth's crust can be safely ignored given experimental precision, at several orders of magnitude [2]. The constant matter approximation is also relevant for NOvA, T2K, HyperK, and JUNO (as well as KamLAND)[1]. For neutrinos passing through varying density profiles, such as the Earth, the NuFast-Earth algorithm from [5] at github.com/PeterDenton/NuFast-Earth, which has its own User Guide, should be used.

Computing neutrino oscillation probabilities in matter is, in principle, straightforward. First, the Hamiltonian governing oscillations from the flavor basis to the flavor basis is:

$$H = \frac{1}{2E}\left[U\begin{pmatrix}0 & & \\ & \Delta m^2_{21} & \\ & & \Delta m^2_{31}\end{pmatrix}U^\dagger + \begin{pmatrix}a & & \\ & 0 & \\ & & 0\end{pmatrix}\right], \tag{1}$$

* pdenton@bnl.gov; 0000-0002-5209-872X

† parke@fnal.gov; 0000-0003-2028-6782

[1] While it might seem that the matter effect is not extremely important for long-baseline reactor neutrinos, it is a subtle effect on a precise measurement [3, 4], and thus must be taken into account.

where $a = 2EV_{\rm CC}$ and $V_{\rm CC} = \sqrt{2}G_F N_e$. Then the oscillation amplitude is

$$\mathcal{A} = \exp(-iHL)\,, \tag{2}$$

and the probability is

$$P_{\alpha\beta} = |\mathcal{A}_{\alpha\beta}|^2\,, \tag{3}$$

where care is to be taken on the order of the α and β indices, see e.g. [6].

Computing this is straightforward and any linear algebra package will suffice. Such a package will perform an eigendecomposition of H when computing the exponential. What we have learned over the years, is that many things that seem to be necessary to compute, do not actually need to be computed. First, we have found that computing the eigenvectors is not necessary. By a careful organization of terms in the amplitude squared (probability), an application of a neutrino specific identity, and a linear algebra identity, it is possible to write the result as an extremely simple function of the eigenvalues, thereby bypassing the fairly expensive eigenvector computation.

Then, given one of the eigenvalues (ideally λ_3 as it is dominantly affected by only one resonance for typical oscillation parameters), the other two can be determined from two (carefully chosen) Cayley-Hamilton conditions. Finally, one can either compute λ_3 exactly (setting `N_Newton<0` in the code) or one can use knowledge of neutrino physics to approximate it (setting `N_Newton=0` in the code) and iteratively improving it rapidly with Newton's method (increasing `N_Newton` to 1, 2, ...). Even without any corrections, the approximation is significantly more accurate than any next generation experiment can achieve, and at two corrections the results are compatible at the double precision level with the exact analytic expression, while still remaining faster.

The code also provides a function to compute neutrino oscillation probabilities in vacuum using the same lessons about structuring the oscillation probability. This is useful for vacuum oscillation calculations, but also provides a benchmark for plausibly the fastest conceivable calculation in matter; given that our fastest calculation is close to this speed, we are confident that the algorithm is about as fast as possible and likely cannot be significantly enhanced.

Finally, new in `v2.0.0`, energies are passed in as vectors and probability matrices returned as a vector of matrices. This allows for reusing of numerous calculations of helpful functions of the six vacuum oscillation parameters that are not affected by the energy or matter potential. The modest overhead increase due to vectorization is rapidly negated at only ~ 2 energy points, and ~ 10s of percent additional speed enhancement is found thereafter.

II. USAGE

The code is designed to be incredibly minimal and easy to use. The key functionality exists in two main functions fully contained within a single file and namespace `Nufast`, but many users will probably only use the matter effect function. The two functions are[2]:

```
void Probability_Matter_LBL(double s12sq, double s13sq, double s23sq, double delta, double Dmsq21, double Dmsq31,
    double L, vector<double> E, double rhoYe, int N_Newton, vector<array<array<double, 3>, 3>> *probs_returned);
void Probability_Vacuum_LBL(double s12sq, double s13sq, double s23sq, double delta, double Dmsq21, double Dmsq31,
    double L, vector<double> E, vector<array<array<double, 3>, 3>> *probs_returned);
```

As arguments, these take in the mixing angles in the usual fashion, as $\sin^2\theta_{ij}$. The complex phase δ is in radians and the Δm^2_{ij}'s are in eV^2. The distance L is in km and the energies are in GeV (although it works for any energy). The algorithm handles the normal (inverted) mass ordering with positive (negative) Δm^2_{31}. For antineutrinos, the algorithm uses negative energies[3]. For the matter effect function ρY_e is the density (g/cc) times the electron fraction; these are typically ~ 3 g/cc for the density and 0.5 for the electron fraction.

The parameter $N_{\rm Newton}$ governs the calculation of one eigenvalue, λ_3, and is the only approximation in the calculation. Any negative value calls the exact analytic expression. A value of zero uses the approximation which is sufficiently accurate in all relevant regions of parameter space for all relevant experimental precisions. Two corrections yields more precision than can be identified with double precision and is still faster than the analytic expression. If a user finds themself needing more than two Newton corrections, either something is going wrong, or they are considering oscillation parameters very far from the best fit points and the analytic expression should be used. Finally, the results are returned as a vector of 3×3 matrices which should be allocated to the same size as the energy vector.

[2] Note that these require vectors and arrays via `#include<vector>` and `#include<array>` and are most easily handled with `using std::vector, std::array;`.

[3] While this choice is not ubiquitous, it reduces the computational overhead of another `if` statement and it also conveniently allows for a single calculation of probabilities for both neutrinos and antineutrinos with one vector of energies.

The first dimension of the vector refers to the energy vector, the second to the initial neutrino flavor, and the third to the final neutrino flavor. So if the energy vector is `E=3,4,5,6` then `probs_returned[2][1][0]` is the $P(\nu_\mu \to \nu_e)$ probability at $E = 5$ GeV. Ideally, the `probs_returned` vector is allocated once and reused for each calculation to avoid unnecessary memory management costs.

Initialization may look like this:

```
int nE = 10;
vector<double> E(nE);
for (int i = 0; i < nE; i++)
   E[i] = i + 1; // creates an array of energies: {1, 2, 3, ... 10} in GeV
vector<array<array<double, 3>, 3>> probs_returned;
probs_returned.resize(nE);
```

Then one can set the oscillation and experimental parameters and call the function:

```
s12sq = 0.31;
s13sq = 0.02;
s23sq = 0.55;
delta = -0.7 * M_PI;
Dmsq21 = 7.5e-5;  // eV^2
Dmsq31 = +2.5e-3; // eV^2
L = 1300; // km
rhoYe = 3 * 0.5; // g/cc
N_Newton = 1;

Probability_Matter_LBL(s12sq, s13sq, s23sq, delta, Dmsq21, Dmsq31, L, E, rhoYe, N_Newton, &probs_returned);
```

The vacuum function call, `Probability_Vacuum_LBL`, operates in the same fashion but without the `rhoYe` or `N_Newton` variables.

III. PERFORMANCE

The performance of `v1.0` of this code, and other algorithms, were benchmarked in the original paper [1]. We highlight some key features of the core algorithm.

1. The approach to eigenvalues and eigenvectors is a significant reduction in computational time over computing the full eigendecomposition.

2. By structuring the oscillation probabilities the way we have, fewer trig functions need to computed. In addition, only three full probabilities need to be constructed in order to compute all nine by using unitarity and CPT invariance along with the structure of the probabilities.

3. The only approximation in the process is for λ_3. This is accurate[4] at the 10^{-5} level (far better than any future oscillation experiment can measure) and improves rapidly with additional Newton corrections: 10^{-10} and 10^{-20} for one or two corrections, respectively.

4. The `NuFast-LBL` algorithm, along with most numerical calculations, benefits from going beyond the default `-O0` optimization flag. We recommend either `-O3`. One can also use `-Ofast` if that is used elsewhere and our tests show no problem with this flag, although there may be problems with it elsewhere.

We show a new speed test including vectorization in figure 1. We note several important features of this result.

- Most importantly, this figure now shows the impact of vectorization of neutrino energies. In particular, we implemented vectorization for our main code, our vacuum code, and the J. Page code (a concept that was presented in that paper, although not for the first time). We have checked and for one energy point the code is slightly slower than the `v1.0` or `v1.1` codes, but this is almost immediately overcome with ultimate speed ups achieved of 32-43%, with even more significant speed ups for the vacuum expression[5].

- The times are, in general, slightly faster than in the paper [1]; this is due to running on a different laptop.

---

[4] Note that this accuracy assumes that the oscillation parameters are "reasonable", notably that s^2_{13} and $\Delta m^2_{21}/|\Delta m^2_{31}|$ are both small.
[5] A comparable speed up of 31% is found for the J. Page code.

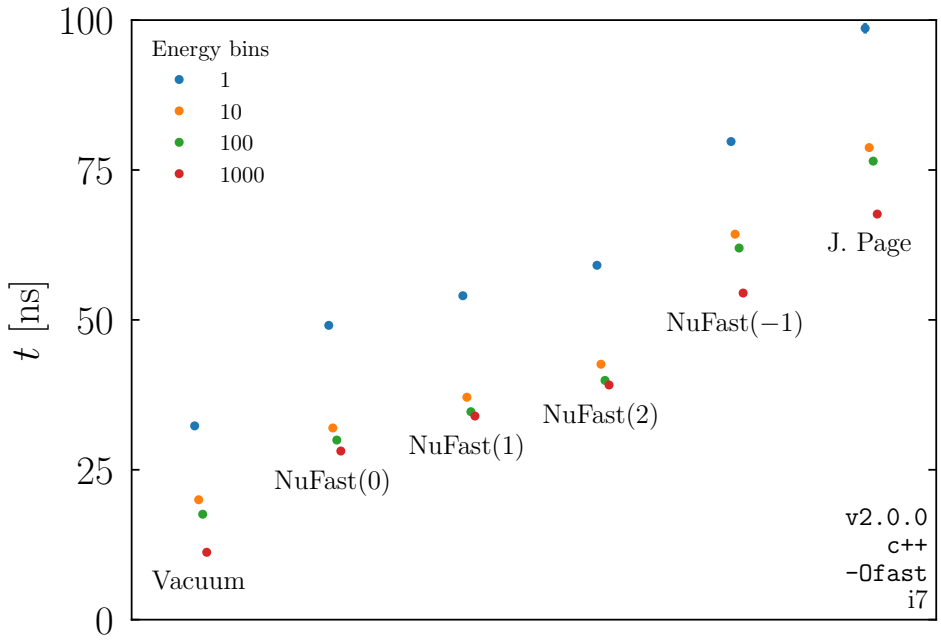

FIG. 1. The time it takes to compute one neutrino oscillation probability for different codes and for different levels of energy vectorization. The probabilities are computed a million times, and this is repeated a thousand times to generate an uncertainty on the mean; the errorbars are plotted but are too small to see. The argument to `NuFast` indicates the number of Newton corrections to the third eigenvalue, with `-1` using the analytic expression.

- We only show the results from so-called aggressive compiler flags `-Ofast`; we tested all the codes with different compiler flags and find that they scale as expected. In particular, `-O3` is essentially identical to `-Ofast`, and `-O0` is a factor of some 10s of percent slower, consistent with previous tests.
- The J. Page data points refer to the code presented in [7].
- We see that the exact analytic expression is 26-36% slower than using `N_Newton=2`.
- We see that the vacuum expression is only 34-60% faster than the `N_Newton=0` expression implying that it is unlikely that we can do significantly better on our calculation of neutrino oscillation probabilities in matter.

IV. COMPILING

No special libraries or flags are needed during compilation. While we do recommend `-O3` or `-Ofast`, they are not strictly necessary.

V. CHANGELOG

A. `v2.0.0`

This is the first major change to `NuFast-LBL` since release. We first highlight the key changes and then discuss them in detail:

1. The energies and oscillation probabilities have been vectorized, and the 3×3 matrix of probabilities are now using `std::arrays`.
2. The density information in ρ and Y_e has been collapsed.
3. The fortran and python code have been deprecated.
4. The addition of this User Guide.

1. *Vectorization*

The code now takes a vector of energies (still in GeV) instead of just a single number. The vector can contain both positive (for neutrinos) and negative (for antineutrinos) numbers in any order. The pointer to the probabilities is now to a vector of 3×3 matrices; the vector should be allocated before passing via `probs_returned.resize(E.size());`, by initializing it as `vector<array<array<double, 3>, 3>> probs_returned(Es.size());`, or equivalent. The 3×3 matrices are now arrays of arrays: `array<array<double, 3>, 3>;` no special action is required for this except the inclusion of the relevant headers.

We note that, while there is some modest increase in the overhead shifting to vectorized energies, a careful numerical analysis shows that the breakeven point is at about 2 energy points. This cost is due to entering into the loop over energies as well as passing vectors. That is, if you call `NuFast-LBL` with a vector of just a single energy then it will be slightly slower than the previous `v1.1`, but at two energy points it is about the same and for any number of energy points greater than that the speed continues to increase. This breakeven point is very similar for the vacuum expression as well as the standard expression for different numbers of N_{Newton} corrections. The numerical fit across a range of energy points finds that the breakeven point is between 1.9 and 2.8 energy grid points, depending on whether the vacuum expression is used, or how many N_{Newton} corrections are used. For any typical problem, especially those that are likely to be the most computationally expensive, we assume that $\mathcal{O}(100)$ or more energy points will be used. If the user is interested in a single energy only (e.g. for a phenomenologists fit to off-axis LBL data) then the modest cost due to energy vectorization should be irrelevant.

2. ρY_e

Another key change is collapsing ρ and Y_e into a single variable called `rhoYe`. This is partly to streamline the code (they only ever appear together as the product) as well as for consistency with the `NuFast-Earth` code. While it is true that some new physics scenarios which may be implemented in the future depend on these parameters separately, prematurely handling those effects is inefficient.

3. *Fortran and python deprecation*

We have decided to deprecate the Fortran and python codes. They can be found in previous versions on github, but will no longer be maintained. This is partially due to the progress of development of the `NuFast-Earth` code in `c++`. It is also due to the fact that we are not aware of anyone using the fortran code and, while the python code has been used, considering the fact that it is naturally much slower than the `c++` code, we encourage users to transition to the `c++` code if speed is needed.

B. `v1.1`

The most important change in `v1.1` is the addition of the exact analytic expression by passing any negative number to `N_Newton`. This was introduced due to some users performing MCMC runs that were sampling the full space of Δm^2_{31} from positive to negative and everywhere inbetween leading to a violation of the perturbative condition requiring $\Delta m^2_{21}/|\Delta m^2_{31}|$ small for the approximation's validity.

[1] P. B. Denton and S. J. Parke, Fast and accurate algorithm for calculating long-baseline neutrino oscillation probabilities with matter effects, Phys. Rev. D **110**, 073005 (2024), arXiv:2405.02400 [hep-ph].

[2] K. J. Kelly and S. J. Parke, Matter Density Profile Shape Effects at DUNE, Phys. Rev. D **98**, 015025 (2018), arXiv:1802.06784 [hep-ph].

[3] Y.-F. Li, Y. Wang, and Z.-z. Xing, Terrestrial matter effects on reactor antineutrino oscillations at JUNO or RENO-50: how small is small?, Chin. Phys. C **40**, 091001 (2016), arXiv:1605.00900 [hep-ph].

[4] A. N. Khan, H. Nunokawa, and S. J. Parke, Why matter effects matter for JUNO, Phys. Lett. B **803**, 135354 (2020), arXiv:1910.12900 [hep-ph].

[5] P. B. Denton and S. J. Parke, Efficient atmospheric, solar, and supernova neutrino propagation through the Earth, Phys. Rev. D **113**, 053002 (2026), arXiv:2511.04735 [hep-ph].

[6] P. B. Denton, Neutrino Oscillations in the Three Flavor Paradigm, (2025), arXiv:2501.08374 [hep-ph].

[7] J. Page, Fast exact algorithm for neutrino oscillation in constant matter density, Comput. Phys. Commun. **300**, 109200 (2024), arXiv:2309.06900 [hep-ph].